

Dimensional Modeling

Monday, June 23, 2025 9:20 AM

1. What is Dimensional Modeling?

Dimensional modeling is a **design technique** used in **data warehouses** to make data easy to understand and fast to query, especially for analytics and reporting.

It organizes data into **facts** (measurable events) and **dimensions** (descriptive context).

2. Why Dimensional Modeling?

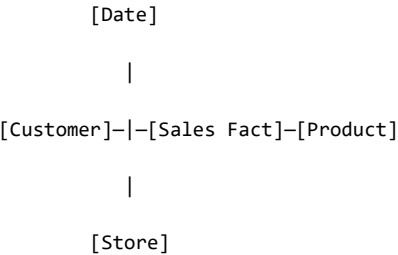
Traditional (OLTP) Modeling	Dimensional Modeling (OLAP)
Optimized for insert/update	Optimized for read/analysis
Complex joins	Simple joins
Normalized structure	Denormalized star/snowflake schemas
Designed for transactions	Designed for reporting & analytics

3. Key Components of Dimensional Modeling

Component	Description
Fact Table	Contains numeric/measurable data (e.g., sales, revenue, count)
Dimension Table	Contains descriptive attributes (e.g., customer name, date, product info)
Grain	Level of detail of the fact (e.g., 1 row = 1 order line)
Measures	Aggregatable numeric data in fact table

4. Star Schema

Definition: A schema where a central fact table is surrounded by dimension tables.



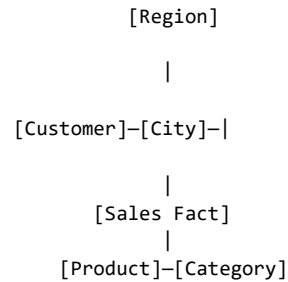
Features:

- Simple structure
- Fast query performance

- Easy for analysts to use

5. Snowflake Schema

Definition: A normalized version of the star schema where dimension tables are split into sub-dimensions.



Features:

- Reduces data redundancy
- Can be more complex to query
- Slightly slower performance than star schema

6. Types of Fact Tables

Type	Description
Transactional	1 row per event (e.g., order, sale)
Snapshot	Periodic summary (e.g., daily inventory levels)
Accumulating	Tracks progress of a process (e.g., from order to delivery milestones)

7. Types of Dimensions

Dimension Type	Description
Conformed Dimension	Shared across multiple fact tables (e.g., Date, Customer)
Role-Playing	Same dimension used for different roles (e.g., Order Date, Ship Date)
Junk Dimension	Combines low-cardinality flags/attributes (e.g., Yes/No flags)
Degenerate Dimension	Exists in fact table but not stored as a separate table (e.g., invoice number)
Slowly Changing Dimension (SCD)	Tracks changes in dimension attributes over time

8. Grain Definition – Why It Matters

Grain = Level of detail in the fact table

Examples:

- “One row per sale”
- “One row per customer per month”

Why it’s important:

- Determines which dimensions you need
- Affects performance and storage
- Must be clearly defined before building schema

9. Dimensional Modeling Process (Step-by-Step)

1. Choose the **business process** (e.g., Order Processing)
2. Declare the **grain** of the fact table
3. Identify the **dimensions**
4. Identify the **measures** (facts)
5. Build the **star** or **snowflake schema**

10. Real-World Example

Business Process: Online Retail Sales

Table	Columns
sales_fact	sale_id, product_id, customer_id, date_id, amount
product_dim	product_id, name, category, brand
customer_dim	customer_id, name, location, email
date_dim	date_id, full_date, day, month, year

11. Benefits of Dimensional Modeling

- Simplifies complex queries
- Supports high-performance analytics
- Aligns with business processes
- Friendly to BI tools like Tableau, Power BI

Slowly Changing Dimensions (SCD)

Types of SCDs

SCD Type 0: Fixed Dimensions (No Changes Allowed)

Definition: Once loaded, the record never changes. Data is treated as constant.

Use Case: When the attribute should never change (e.g., Date of Birth, Employee Start Date).

Example:

Customer_ID	Name	DOB
-------------	------	-----

1001	Daniel	1995-01-01
------	--------	------------

If DOB changes by mistake in the source system, the warehouse will ignore it.

SCD Type 1: Overwrite the Old Data

Definition: The old value is overwritten with the new one. No history is preserved.

Use Case: When maintaining historical data is not important (e.g., correcting spelling mistakes).

Example:

Before change:

Customer_ID	City
1001	Chicago

After change:

Customer_ID	City
1001	Houston

Only the latest city is stored.

SCD Type 2: Add a New Row (Full History Tracking)

Definition: New row is inserted for every change. History is preserved by tracking start and end dates (or active flags).

Use Case: Critical for historical analysis (e.g., customer moved cities, and you want to know where they lived during each purchase).

Example:

Customer_ID	Name	City	Start_Date	End_Date	Current_Flag
1001	Daniel	Chicago	2021-01-01	2022-06-30	N
1001	Daniel	Houston	2022-07-01	NULL	Y

Allows tracking **who** the customer was and **where** at any point in time.

SCD Type 3: Add New Columns (Limited History Tracking)

Definition: Add a new column to store the previous value (usually only one previous state is stored).

Use Case: When only limited history is needed (e.g., previous and current manager, region).

Example:

Customer_ID	Name	Current_City	Previous_City
1001	Daniel	Houston	Chicago

Only one previous value is preserved. Older history is lost.

SCD Type 4: Historical Table (Separate History Table)

Definition: Maintain the current record in the main dimension table, and store full history in a separate historical table.

Use Case: When performance and storage are concerns, or when separating current and historical views is needed.

Example:

customer_dim table (current):

Customer_ID	Name	City
1001	Daniel	Houston

customer_history table:

Customer_ID	Name	City	Change_Date
1001	Daniel	Chicago	2021-01-01
1001	Daniel	Houston	2022-07-01

SCD Type Summary Table

SCD Type	Method	Tracks Full History?	Common Use Case
Type 0	No change	No	Immutable attributes
Type 1	Overwrite	No	Minor updates (e.g., typo fix)
Type 2	New row with versioning	Yes	Customer or product changes
Type 3	Add new columns	Partial (1 prev)	Limited history needed
Type 4	Separate history table	Yes	Separate current vs history, audit logs

What is a Fact Table?

A **fact table** is the central table in a star or snowflake schema in a data warehouse. It stores **quantitative data (measures)** for analysis and is surrounded by **dimension tables**.

Fact Table Characteristics

- **Contains:** Numeric values (facts), foreign keys to dimensions
- **Grain:** The level of detail (e.g., daily sales per product per store)
- **Grows rapidly:** As business transactions increase
- **Supports:** Aggregations, slicing and dicing, trends, KPIs

Types of Fact Tables

1. Transactional Fact Table

- **Description:** Captures individual, atomic transactions (e.g., purchases, payments).
- **Grain:** One row per event (e.g., per order, per click).
- **Example:**

Transaction_ID	Date_ID	Product_ID	Store_ID	Quantity	Sales_Amount
TX1001	20240101	P001	S01	2	50.00

- **Use Case:** Sales, Orders, Shipments

2. Snapshot Fact Table

- **Description:** Captures data at regular time intervals (e.g., daily inventory levels).
- **Grain:** One row per period per dimension.
- **Example:**

Snapshot_Date	Product_ID	Store_ID	Inventory_Level
20240101	P001	S01	120

- **Use Case:** Periodic reports, account balances, stock levels

3. Accumulating Snapshot Fact Table

- **Description:** Tracks the lifecycle of a process or workflow.
- **Grain:** One row per process instance, with timestamps of each step.
- **Example:**

Order_ID	Order_Date	Ship_Date	Delivery_Date	Return_Date
1234	20240101	20240103	20240106	NULL

- **Use Case:** Order-to-cash, Claims processing, Ticket lifecycle

4. Factless Fact Table

- **Description:** Contains only dimension keys and **no measurable facts**.
- **Use Case:** To track events or coverage (e.g., student course enrollments, promotional eligibility)
- **Types:**
- **Event tracking** (e.g., page views, visits)
- **Coverage** (e.g., which product is available in which store)

Example: Event Tracking

Student_ID	Course_ID	Enroll_Date
S101	C001	20240105

Fact Table Design Best Practices

1. Define the Grain Clearly

- Example: "One row per product per day per store"

- Impacts what dimensions and facts you include

2. Use Surrogate Keys

- Replace natural keys with system-generated integer IDs
- Faster joins, avoids changes in source system

3. Avoid NULLs in Foreign Keys

- If needed, use a **dummy dimension** (e.g., 'Unknown', 'Not Applicable')

4. Time Dimension is Mandatory

- Always include a **date or timestamp key**
- Supports time-series analytics (daily, monthly, YTD, MTD, etc.)

5. Use Additive, Semi-additive, and Non-additive Facts Properly

Type	Description	Example
Additive	Can be summed across all dimensions	Sales, Quantity
Semi-additive	Can be summed across some dimensions	Account balance (by region but not by time)
Non-additive	Cannot be summed at all	Ratios, Percentages

6. Include Degenerate Dimensions if Needed

- Textual data with no separate dimension (e.g., Invoice ID)
- Useful for filtering or grouping

7. Audit Columns Help

- Include created_at, updated_at, batch_id, etc., for debugging and data lineage

8. Partitioning and Compression

- Partition by date (especially on large transactional tables)
- Use efficient file formats like **Parquet** and **ORC**
- Enable **ZSTD/Snappy compression** for storage and performance